密码锁实验

王珺桥 2024010694 计42

实验代码

代码块

```
module Locker(
    input wire [3:0] Code,
    input wire Mode,
    input wire CLK,
    input wire RST,
    output reg Unlock,
    output reg Err,
    output reg alert
    );

typedef enum {init, set0, set1, set2, set3, val0, val1, val2, val3, open} state_t; // init 初始状态 set0-set3 设置密码的四步 val0-val3 验证密码的四步
state_t state;

reg [3:0] admin_password[0:3] = '{4'd0, 4'd5, 4'd0, 4'd1}; // 默认管理员密码
reg [3:0] user_password[0:3] = '{4'd1, 4'd2, 4'd3, 4'd4};  // 初始默认密码为1234，存储用户密码
shortint wrongtimes = 0;
logic admin_wrong;
logic isWrong;

reg alert_reg = 0;
assign alert = alert_reg;

always_ff @(posedge CLK or posedge RST) begin
    if (RST) begin // 复位
        isWrong <= 0;
        state <= init;
        Unlock <= 0;
        Err <= 0;
        admin_wrong <= 0;
//        alert <= 0;
//        wrongtimes <= 0;
    end
    else begin
        case (state)
```

```
35              init: begin //选择Mode的状态
36                  case (Mode)
37                      1'b0: begin
38                          state <= set0; // Mode = 0, 为设置密码模式
39                          Unlock <= 1;   //设置密码时默认为开锁
40                      end
41                      1'b1: begin
42                          state <= val0; // Mode = 1, 为验证密码模式
43                          Unlock <= 0;
44                      end
45                  endcase
46                  Err <= 0;
47              end
48              set0: begin
49                  user_password[0] <= Code;
50                  state <= set1;
51              end
52              set1: begin
53                  user_password[1] <= Code;
54                  state <= set2;
55              end
56              set2: begin
57                  user_password[2] <= Code;
58                  state <= set3;
59              end
60              set3: begin
61                  user_password[3] <= Code;
62                  state <= open; // 密码设置完成
63                  Unlock <= 0;   //上锁
64              end
65              val0: begin // 无警报验证用户密码，有警报验证管理员密码
66                  if ((~alert && Code != user_password[0]) || (alert && Code !=
    admin_password[0]) || admin_wrong) begin
67                      isWrong <= 1;
68                  end
69                  state <= val1;
70              end
71              val1: begin
72                  if ((~alert && Code != user_password[1]) || (alert && Code !=
    admin_password[1]) || admin_wrong) begin
73                      isWrong <= 1;
74                  end
75                  state <= val2;
76              end
77              val2: begin
78                  if ((~alert && Code != user_password[2]) || (alert && Code !=
    admin_password[2]) || admin_wrong) begin
```

```
                    isWrong <= 1;
                end
                state <= val3;
            end
            val3: begin
                if ((~alert && Code != user_password[3]) || (alert && Code != admin_password[3]) || admin_wrong) begin
                    isWrong <= 1;
                end
                if(isWrong == 1) begin
                   if (~alert) begin
                       isWrong <= 0;
                       state <= init;
                       Unlock <= 0; //上锁
                       Err <= 1;
                       wrongtimes++;
                       if (wrongtimes >= 3) alert <= 1;
                   end
                   else begin
                       isWrong <= 0;
                       state <= init;
                       Err <= 1;
                       Unlock <= 0;
                   end
                end
                else begin //密码验证正确
                    state <= open;
                    wrongtimes <= 0;
                    Err <= 0;
                    Unlock <= 1;
                    if (alert == 1) begin // 报警之后管理员开锁
                        alert <= 0;
                        Unlock <= 1;
                    end
                end
            end
            open: begin
                if(alert == 1)begin
                    Unlock <= 0;
                end
                else begin
                    Unlock <= 0;
                    state <= init;
                end
            end
            default: state <= init;
        endcase
```

```
125         end
126     end
127 
128     endmodule
```

本次串行密码锁实验我总共设置了10种状态

状态	功能
init	初始状态，选择模式
set0	输入密码第1位
set1	输入密码第2位
set2	输入密码第3位
set3	输入密码第4位
val0	验证第1位
val1	验证第2位
val2	验证第3位
val3	验证第4位并判断
open	开锁状态

管脚绑定

代码块

```
1   #CLK input
2   set_property -dict {PACKAGE_PIN J19 IOSTANDARD LVCMOS33} [get_ports CLK];
    #CLK接插孔
3   
4   #RST input
5   set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports RST];
    #IO0接插孔
6   
7   # Code input
8   set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports Code[3]];
       #IO1接插孔
9   set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports Code[2]];
       #IO2接插孔
10  set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports Code[1]];
       #IO3接插孔
11  set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports Code[0]];
       #IO4接插孔
12  
13  # Mode input
```

```
set_property -dict {PACKAGE_PIN W21 IOSTANDARD LVCMOS33} [get_ports Mode];
#IO11接插孔

# Unlock output
set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Unlock];
   #IO16接插孔

# Err output
set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports Err];
#IO17接插孔

# alert output
set_property -dict {PACKAGE_PIN AA20 IOSTANDARD LVCMOS33} [get_ports alert];
  #IO18接插孔

# required if touch button used as manual clock source
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets CLK_IBUF]

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

课堂验收

首先设置用户密码为1234，然后切换至验证模式输入相同密码，Unlock 指示灯点亮；输入错误密码时，Err 指示灯点亮；连续三次输入错误密码后，Alert 指示灯点亮并进入锁定状态。实际测试结果与Vivado仿真结果一致。

实验仿真

代码块

```
module Locker_tb;

  // 输入
  reg [3:0] Code;
  reg Mode;
  reg CLK;
  reg RST;

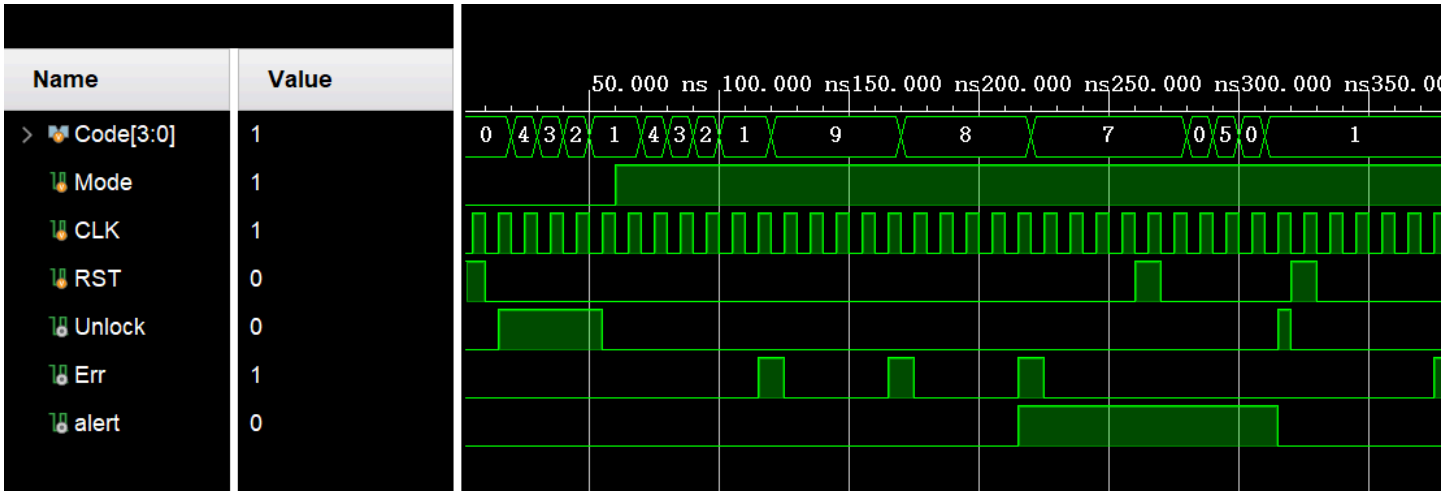
  // 输出
  wire Unlock;
  wire Err;
  wire alert;

  // 实例化被测试模块
  Locker uut (
```

```
16      .Code(Code),
17      .Mode(Mode),
18      .CLK(CLK),
19      .RST(RST),
20      .Unlock(Unlock),
21      .Err(Err),
22      .alert(alert)
23    );
24
25    // 时钟生成
26    always #5 CLK = ~CLK;
27
28    task reset();
29      begin
30        RST = 1; #10;
31        RST = 0;
32      end
33    endtask
34
35    task set_user_password(input [3:0] p0, p1, p2, p3);
36      begin
37        Mode = 0; #10; // 进入设置模式
38        Code = p0; #10;
39        Code = p1; #10;
40        Code = p2; #10;
41        Code = p3; #10;
42      end
43    endtask
44
45    task validate_password(input [3:0] p0, p1, p2, p3);
46      begin
47        Mode = 1; #10; // 进入验证模式
48        Code = p0; #10;
49        Code = p1; #10;
50        Code = p2; #10;
51        Code = p3; #10;
52      end
53    endtask
54
55    initial begin
56      $display("Start Simulation");
57      CLK = 0;
58      Code = 0;
59      Mode = 0;
60      RST = 0;
61
62      // 初始化系统
```

```
63        reset();
64
65        // 设置新用户密码为 1,2,3,4
66        set_user_password(1, 2, 3, 4);
67
68        // 正确验证密码
69        validate_password(1, 2, 3, 4);
70
71        // 三次错误输入触发报警
72        validate_password(9, 9, 9, 9);
73        validate_password(8, 8, 8, 8);
74        validate_password(7, 7, 7, 7);
75
76        reset(); // 重置系统，检查警报是否会解除
77
78        // 管理员输入正确密码解锁：0,5,0,1
79        validate_password(0, 5, 0, 1);
80
81        reset();
82
83        $display("Simulation Complete");
84        #50 $finish;
85      end
86
87  endmodule
```

仿真结果



实验中遇到的问题

最开始测试时，发现输入四位密码按clk时，指示灯有时候亮有时候不亮。对clk的可用性和指示灯都进行测试后，修改代码使得串行输入密码时需要严格按照输入一位密码后按一次clk的方式操作。

在进行实验仿真时，由于alert没有在源代码中初始化，所以在仿真时会出现错误，但是如果使用initial初始化仿真会报错初始化两遍，考虑到功能又不可以在仿真时触发，所以我引入了一个alert_reg寄存器来帮助alert初始化，避免了这个问题。可见在错误后reset但是不会解除警报。